README

ota-artifact-validator

Field	Value
Revision	2
Created	2026-06-07
Status	implemented
Part of	Helix OTA
Module path	`github.com/HelixDevelopment/ota-artifact-validator`
Language	go (1.26)
License	Apache-2.0

Overview

`ota-artifact-validator` (package `otavalidator`) implements the server-side OTA artifact upload validation pipeline as an **ordered, fail-fast decision table** (stages S2..S6). It is a **pure library**: it operates on bytes / `io.Reader` / interface ports only and performs **no disk, network, HTTP, or database access** (`verdict.go`). The caller (the control-plane upload handler, or CI) supplies all inputs and implements the lookup ports, keeping the validator decoupled and independently testable.

S1 (ZIP/structure parsing) is out of scope for this pure-bytes library — it is handled by the upload handler's archive reader — so the implemented stages are:

Stage	Constant	Checks
S2	`StageHash`	streamed SHA-256 vs the external hash file
S3	`StageSignature`	detached **ed25519** signature over the S2 digest
S4	`StageVersion`	version monotonicity (no downgrade / duplicate)
S5	`StageTarget`	target compatibility (`os_type` / `board` known + supported)
S6	`StageMetadata`	

Stage	Constant	Checks
		metadata sanity + consistency with the S2 digest

Each stage returns a typed `Verdict` carrying a stable `RejectCode` (no key material). The top-level `Validate` runs the pipeline and returns the first rejecting verdict.

Public API

Pipeline (`pipeline.go`)

- `Input` — pure data + ports: `Artifact io.Reader`, `HashFile string`, `PublicKey ed25519.PublicKey`, `Signature []byte`, `CurrentVersion string`, `Meta otaprotocol.ArtifactMeta`, `VersionComparator`, `TargetPolicy`.
- `Result` — `Final Verdict`, `Verdicts []Verdict` (per-stage audit trail, in order), `ComputedSHA256 string`; `Accepted() bool`.
- `Validate(Input) Result` — runs the ordered, fail-fast S2..S6 pipeline.

Stage functions (`stages.go`) — composable, usable individually

- `ValidateHash(r io.Reader, expectedHashFile string) (Verdict, string)` — S2; returns the verdict and the computed lowercase-hex digest. Tolerates the coreutils `"<hex>  name"` hash-file form.
- `ValidateSignature(digestHex string, pubKey ed25519.PublicKey, sig []byte) Verdict` — S3; verifies the detached ed25519 signature over the S2 digest.
- `ValidateVersion(declared, currentVersion string, cmp VersionComparator) Verdict` — S4; declared must be strictly greater than the latest published (empty current = no prior release passes).
- `ValidateTarget(os otaprotocol.OSType, board string, policy TargetPolicy) Verdict` — S5.
- `ValidateMetadata(meta otaprotocol.ArtifactMeta, computedDigest string) Verdict` — S6; delegates field checks to `otaprotocol.ValidateArtifactMeta` and cross-checks the digest.

Ports & policies

- `VersionComparator` interface (`Compare(a, b string) (int, error)`) and `VersionComparatorFunc` adapter (`stages.go`).
- `CompareDotted(a, b string) (int, error)` — the default dotted-numeric comparator (`1.10.0 > 1.9.0`, tolerant leading `v`, zero-padded component counts) (`version.go`).
- `TargetPolicy` interface (`Known`, `Supported`) (`stages.go`).
- `StaticTargetPolicy` + `NewStaticTargetPolicy(known, supported []TargetKey)` + `TargetKey{OSType, Board}` — an in-memory policy where a supported target is always known (`target.go`).

Verdicts & codes (`verdict.go`)

- `Verdict` struct (`Passed`, `Stage`, `Code`, `Message`) with `IsReject() bool` and `String()`.
- `Stage` constants: `StageHash`, `StageSignature`, `StageVersion`, `StageTarget`, `StageMetadata`.
- `RejectCode` constants per stage, e.g. `RejectHashMismatch`, `RejectSignatureInvalid`, `RejectNotMonotonic`, `RejectDuplicateVersion`, `RejectTargetUnsupported`, `RejectMetadataInconsistent` (full set in `verdict.go`).

Usage

```go
package main

import (
    "crypto/ed25519"
    "fmt"
    "strings"

    otaprotocol "github.com/HelixDevelopment/ota-protocol"
    otavalidator "github.com/HelixDevelopment/ota-artifact-
        validator"
)

func main() {
    artifact := []byte("payload bytes")
    // (in real use, hashFile/signature/pubKey come from the
        upload + trusted config)

    policy := otavalidator.NewStaticTargetPolicy(
        nil,
        []otavalidator.TargetKey{{OSType: otaprotocol.OSAndroid,
        Board: "rk3588"}},
    )

    res := otavalidator.Validate(otavalidator.Input{
        Artifact:       strings.NewReader(string(artifact)),
        HashFile:       "<64-char-sha256-hex>",
        PublicKey:      ed25519.PublicKey(make([]byte,
        ed25519.PublicKeySize)),
        Signature:      []byte("detached-sig"),
        CurrentVersion: "1.1.0",
        Meta: otaprotocol.ArtifactMeta{
            Version: "1.2.0", OSType: otaprotocol.OSAndroid,
        Board: "rk3588",
            Size: 13, SHA256: "<64-char-sha256-hex>", Signature:
        "sig",
        },
        TargetPolicy: policy,
    })
```

```
    fmt.Println(res.Accepted(), res.Final.Code,
        res.ComputedSHA256)
}
```

Testing

```
cd submodules/ota-artifact-validator
go vet ./...
go test ./...
```

The suite (`validator_test.go`) covers: the full pipeline incl. **fail-fast** ordering (`TestValidate`, `TestFailFast`); each stage in isolation (`TestValidateHash` — bare + coreutils hash-file forms, mismatch; `TestValidateSignature` — missing/invalid key/scope; `TestValidateVersion` + `TestValidateVersionCustomComparatorError` — monotonic/duplicate/downgrade; `TestValidateTarget` — undeclared/unknown/unsupported; `TestValidateMetadata` — incomplete + digest-inconsistent); the dotted-version comparator (`TestCompareDotted`); and verdict rendering (`TestVerdictString`).

Reusable building brick

This is a **reusable, independently versioned** Helix OTA building brick (HelixConstitution §11.4.28 — submodules-as-equal-codebase). Consume it via its module path `github.com/HelixDevelopment/ota-artifact-validator`. It is OS-aware via the injected `TargetPolicy` / `VersionComparator` ports and carries no transport or DB, so it is reusable by any artifact-intake path or CI. Universal constitution rules are inherited via this repo's `CLAUDE.md` / `AGENTS.md` (`## INHERITED FROM Helix Constitution`).

Mirrors

- GitHub: https://github.com/HelixDevelopment/ota-artifact-validator
- GitLab: https://gitlab.com/helixdevelopment1/ota-artifact-validator